
Finite-Support Convolutional Likelihoods for Aggregate-Supervised Multiple-Instance Learning

Anonymous Author(s)

Affiliation

Address

email

Abstract

Aggregate-supervised multiple-instance learning observes a label for each bag but not the labels of individual instances. Exact likelihoods are known for standard Bernoulli count labels, but many aggregate labels involve more general finite supports, including signed counts with cancellation and integer-valued digit sums. We introduce FS-Conv, a backbone-agnostic finite-support convolutional likelihood layer that maps per-instance contribution PMFs to the exact model-implied aggregate distribution by discrete convolution. FS-Conv strictly generalizes the exact Bernoulli count likelihood underlying recent count-based weak-supervision methods: the $\{0, 1\}$ support recovers ordinary count loss, while signed and integer-valued supports yield exact likelihoods for cancellation and digit-sum supervision. Across aggregate-supervised prediction tasks, FS-Conv improves hidden-instance recovery most clearly when the observed aggregate is poorly captured by a mean or proportion alone.

1 Introduction

Multiple-instance learning (MIL) studies settings where supervision is observed for a bag while the labels of individual instances remain latent. Classical binary MIL uses a positive bag label to indicate that at least one instance is positive. Many aggregate-supervision settings provide more structured labels: the number of positive instances, a class histogram, a signed count, or a scalar integer-valued sum. Similar aggregate labels arise in algebraic combinatorics and mathematical physics, where latent contributions often have combinatorial or algebraic structure (Appendix B). The central problem is to learn an instance-level predictor from such aggregate observations without using instance labels during training.

FS-Conv is a finite-support convolutional likelihood layer for aggregate-supervised MIL. Each instance x_i is mapped to a probability mass function over a finite set of integer-valued contributions. The layer returns the mathematically implied distribution of the observed aggregate label by convolving the instance-level atomic PMFs. For Bernoulli contributions this recovers the standard count probability used in prior count-loss work. The same construction also gives exact likelihoods for signed counts, digit-sum labels, and one-vs-rest class-count objectives. We refer to this family as finite-support aggregate likelihoods: the atomic support size and contribution values define the aggregate-supervision task, while FS-Conv is unchanged. The formulation separates the instance model from the aggregate likelihood: a backbone model outputs instance-level scores, and the likelihood layer converts these scores into finite-support atomic PMFs before computing the aggregate likelihood. The method is therefore not tied to a particular architecture. Changing the aggregate label changes only the contribution map and aggregate support; the instance backbone, optimizer, and aggregation rule remain unchanged.

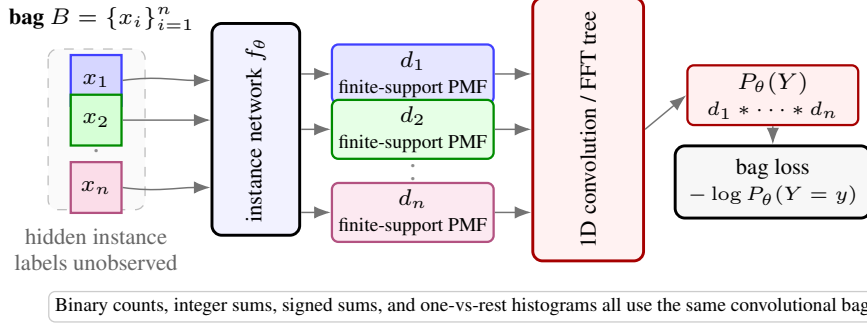


Figure 1: Convolutional aggregate pipeline. The instance network emits finite-support atomic PMFs d_i for each instance. A deterministic convolutional layer computes the aggregate PMF $P_\theta(Y)$, and the observed bag label selects the likelihood term used for training.

This view also makes the novelty boundary explicit. Binary Bernoulli counts recover existing count-loss objectives, and one-vs-rest class-count likelihoods are closely related to the count-likelihood component of LLP-PVC. Our focus is the finite-support likelihood layer that covers these cases while also extending to signed and integer-valued sum labels. FS-Conv is thus a finite-support generalization of the exact count-likelihood component used in recent state-of-the-art count-based weak-supervision and LLP methods: $\{0, 1\}$ atoms recover ordinary count loss, classwise one-vs-rest atoms recover the LLP-PVC count-likelihood component, and signed or integer-valued atoms extend the same rule beyond Bernoulli counts.

Our contributions are:

- We show that exact Bernoulli count likelihoods used in recent count-based weak-supervision methods are the $\{0, 1\}$ instance of a broader finite-support convolutional likelihood family.
- We extend this count-likelihood component to signed counts, digit-sum labels, and one-vs-rest class-count objectives by changing only the finite-support contribution map.
- We implement practical convolutional backends, including grouped signed convolution and a GPU FFT-tree backend that parallelizes large multiclass one-vs-rest PMFs beyond CPU dynamic programming.
- We evaluate hidden-instance recovery across Bernoulli count, signed-count, digit-sum, SVHN sum, and one-vs-rest class-count settings.

Empirically, the largest improvements occur for aggregate types where expectation matching discards information, especially digit sums and signed counts. The CIFAR-10 comparison shows that one-vs-rest count likelihood achieves high instance accuracy when paired with ImageNet-pretrained features, but the fixed-instance-budget result attributes the large-bag improvement to greater sampled image exposure rather than to bag size alone.

2 Related work

Multiple-instance learning and count loss. Classical multiple-instance learning supervises bags rather than instances [8, 19, 1, 6]; neural and attention MIL learn bag predictors end-to-end [28, 11]. These MIL models usually target binary bag-presence labels, such as whether at least one positive instance is present, whereas our aggregate labels are exact integer counts, signed sums, or integer-valued sums. Count-based aggregate supervision gives a stronger label by observing the number of positive instances. Prior count-loss methods compute exact Bernoulli count probabilities and have achieved state-of-the-art or highly competitive results in count-based weak supervision [23]; related count supervision appears in temporal localization [21]. FS-Conv contains these objectives as the $\{0, 1\}$ special case and lifts the same exact-likelihood mechanism to signed cancellation and integer-valued sums.

71 **Learning from label proportions.** Learning from label proportions observes class proportions or
 72 histograms over bags [20, 31], with probabilistic, deep, pseudo-labeling, consistency, and two-stage
 73 variants [25, 3, 10, 9, 14, 26, 15, 30, 16] and recent theoretical analyses [22, 32, 4, 29, 2, 5]. LLP-
 74 PVC [17] reframes label proportions as proportional-value classification and uses efficient count-
 75 likelihood computation. Our classwise count objective recovers this likelihood component, not the
 76 entire LLP-PVC method or a new full joint histogram model.

77 **Efficient aggregate inference.** Dynamic programming computes Bernoulli count probabilities
 78 exactly but is sequential and can be expensive for large bags and many classes [17]. The same
 79 finite-support likelihood can instead be evaluated by direct convolution or FFT polynomial multi-
 80 plication [7]. We use sequential convolution for moderate aggregate supports and a GPU FFT-tree
 81 backend for large one-vs-rest count PMFs; this changes the backend, not the statistical objective.

82 3 Problem setup

83 We study aggregate-supervised bags of instances. A training example is a bag $B = (x_1, \dots, x_n)$
 84 together with an observed aggregate label y . The instance-level variables that explain y are hidden
 85 during training and are used only for evaluation. The aggregate label may be a Bernoulli count, a
 86 signed count, an integer-valued sum, or a class histogram. In all cases, each instance contributes
 87 a finite-support discrete value, and the observed label is the sum of these contributions. We define
 88 FS-Conv as one likelihood layer for this structure, so different aggregate-supervision tasks require
 89 only different atomic supports and contribution maps.

90 The unifying object is the *atomic support*. For a one-dimensional task, choose a finite set

$$\mathcal{A} = \{v_0, \dots, v_{S-1}\} \subset \mathbb{Z}, \quad (1)$$

91 where $S = |\mathcal{A}|$ is the atomic support size. A backbone model produces scores for each instance,
 92 and a contribution map turns those scores into an atomic PMF over $\mathcal{A}$. Changing $\mathcal{A}$ changes the
 93 aggregate-supervision task while leaving the aggregate-prediction algorithm unchanged: $S = 2$
 94 gives Bernoulli counts, $S = 10$ gives digit sums over $\{0, \dots, 9\}$, and $S = 3$ gives signed atoms over
 95 $\{-1, 0, +1\}$. For bag size n and contiguous support $\{0, \dots, S-1\}$, the aggregate PMF has length

$$T = n(S-1) + 1. \quad (2)$$

96 Signed supports use the same principle with an offset, and more general atoms can use instance-
 97 dependent supports.

98 Let C_i denote the latent contribution of instance x_i . The contribution may also depend on observed
 99 side information a_i , such as a known sign in signed-count experiments. We assume

$$C_i \in \mathcal{S}_i \subset \mathbb{Z}^d, \quad |\mathcal{S}_i| < \infty, \quad (3)$$

100 and the observed aggregate is

$$Y = \sum_{i=1}^n C_i. \quad (4)$$

101 Most experiments use $d = 1$. For histograms, d may be the number of classes, although our
 102 scalable implementation uses one-dimensional one-vs-rest count likelihoods. The learner observes
 103 $(x_{1:n}, a_{1:n}, y)$ when side information is present, but never observes the individual C_i values during
 104 training. When discussing individual scalar tasks, we write Y_{count} for ordinary nonnegative counts,
 105 Y_{sum} for integer-valued sums such as digit sums, and $Y_{\pm}$ for signed counts. These are task-specific
 106 instances of the generic aggregate Y in Eq. (4).

107 **Definition 1** (FS-Conv finite-support convolutional likelihood layer). *Let f_θ be any instance model,*
 108 *and let Φ be a contribution map that converts the output of f_θ on instance x_i and observed side*
 109 *information a_i into an atomic PMF*

$$d_i = \Phi(f_\theta(x_i), a_i) \in \Delta(\mathcal{S}_i), \quad \mathcal{S}_i \subset \mathbb{Z}^d, \quad |\mathcal{S}_i| < \infty. \quad (5)$$

110 *For conditionally independent latent contributions $C_i \sim d_i$, FS-Conv is the deterministic map*

$$\text{Agg}(d_1, \dots, d_n)(y) = (d_1 * \dots * d_n)(y) = \sum_{\substack{c_1 \in \mathcal{S}_1, \dots, c_n \in \mathcal{S}_n \\ c_1 + \dots + c_n = y}} \prod_{i=1}^n d_i(c_i), \quad (6)$$

111 *which assigns the exact probability mass of the aggregate $Y = \sum_i C_i$ to value y .*

112 For Bernoulli atoms $d_i = [1 - p_i, p_i]$ on $\{0, 1\}$, FS-Conv recovers the count-loss probability

$$\Pr_{\theta}(Y = k) = (d_1 * \dots * d_n)(k) = [t^k] \prod_{i=1}^n ((1 - p_i) + p_i t), \quad (7)$$

113 the Poisson-binomial probability optimized by Bernoulli count-loss methods. Under identical in-
 114 stance probabilities, FS-Conv and CountLoss define the same loss and gradients up to backend dif-
 115 ferences. Applying the same equation independently to each class with atoms $[1 - p_{ic}, p_{ic}]$ recovers
 116 the one-vs-rest count-likelihood component used in LLP-PVC-style objectives, but not a normalized
 117 full joint multiclass histogram likelihood.

118 Definition 1 separates four choices often coupled in task-specific aggregate-supervision losses:

- 119 • *Backbone independence.* The network supplies atomic PMFs; the likelihood layer is un-
 120 changed across MLPs, CNNs, transformers, or pretrained encoders.
- 121 • *Contribution-map independence.* Different aggregate-supervision tasks are specified by the
 122 contribution map and finite support, not by changing the aggregation rule.
- 123 • *Backend independence.* The returned PMF supports NLL training, while its computation
 124 can be implemented by dynamic programming, direct convolution, grouped convolution,
 125 or FFT-tree multiplication.
- 126 • *Conditional-independence scope.* The PMF is exact under conditionally independent latent
 127 contributions; otherwise it is a structured aggregate-likelihood approximation.

128 We train with the aggregate negative log likelihood $-\log \text{Agg}(d_1, \dots, d_n)(y)$ for the observed label
 129 y .

130 Once the atomic contribution distributions d_i are predicted, the aggregate-label distribution is fixed
 131 by convolution under the conditional-independence model. Tasks differ in the support S_i , the contri-
 132 bution map Φ , and which side information is observed; concrete kernels are summarized in Table 1.
 133 Bernoulli count supervision is the simplest case of Eq. (4), while signed counts, integer-valued sums,
 134 multiplicity counts, and one-vs-rest class counts change the finite-support kernel but keep the same
 135 likelihood layer. In our experiments, bags are sampled on the fly with replacement from the corre-
 136 sponding train or test split. Thus an image may appear in multiple bags, but train and test bags are
 137 always drawn from disjoint dataset splits.

138 4 Convolutional bag prediction

139 4.1 Aggregate likelihood

140 The instance network and contribution map emit the atomic PMFs d_i used in Definition 1. For
 141 one-dimensional contributions, this is a categorical distribution over a finite integer support:

$$d_i(k) = p_{\theta}(z_i = k \mid x_i), \quad k \in \{a, \dots, b\}. \quad (8)$$

142 The exact aggregate-label likelihood is FS-Conv applied to these atomic PMFs:

$$p_{\theta}(Y = y \mid x_{1:n}) = \text{Agg}(d_1, \dots, d_n)(y) = (d_1 * d_2 * \dots * d_n)(y). \quad (9)$$

143 Training minimizes the negative log likelihood

$$\mathcal{L}_{\text{agg}}(\theta) = -\log p_{\theta}(Y = y \mid x_{1:n}). \quad (10)$$

144 The support of the aggregate PMF is finite and contiguous:

$$\text{supp}(Y) = \{na, na + 1, \dots, nb\}. \quad (11)$$

145 Equivalently, if each atom has contiguous support size $S = b - a + 1$, then the aggregate PMF has
 146 width $T = n(S - 1) + 1$ after shifting by na . For scalar aggregate labels, all experiments evaluate the
 147 exact model-implied likelihood over this support; approximation enters through the neural instance
 148 model rather than through sampled latent assignments.

Table 1: Task kernels used by FS-Conv. The support order is the array order for the atomic PMF.

Task	Aggregate label	Atomic kernel	Support
Binary count	$Y = \sum_i z_i$	$[1 - p_i, p_i]$	$\{0, 1\}$
Signed count	$Y_{\pm} = \sum_i s_i z_i$	$\kappa_i^{(+)} = [0, 1 - p_i, p_i]; \kappa_i^{(-)} = [p_i, 1 - p_i, 0]$	$\{-1, 0, +1\}$
Integer sum	$Y_{\text{sum}} = \sum_i z_i$	$[p_{i0}, \dots, p_{i,K-1}]$	$\{0, \dots, K - 1\}$
Classwise composite count	$h_c = \sum_i z_{ic}$	$[1 - p_{ic}, p_{ic}]$ for each class c	$\{0, 1\}$ per class

Two immediate consequences. First, when the atomic support is Bernoulli, the convolutional likelihood is exactly the coefficient of the polynomial $\prod_i ((1 - p_i) + p_i t)$ and therefore recovers the standard count probability. Second, for any finite support, the likelihood uses the entire induced aggregate distribution, not only the mean aggregate value. Thus two instance predictors with the same expected sum can receive different likelihoods if they assign different probability mass to the observed aggregate. This is the distinction exploited by the digit-sum and signed-count experiments.

4.2 Training objective

For each minibatch, the instance model and task-specific contribution map produce atomic PMFs for every instance in every bag. FS-Conv then maps these atoms deterministically to an exact PMF over the observed aggregate label. This separation makes the objective backbone-agnostic: an MLP, CNN, transformer, or pretrained image encoder can be used as long as it produces per-instance scores from which Φ can form the required finite-support PMFs. It also makes the aggregate-supervision family modular: Bernoulli counts, signed counts, integer-valued sums, and one-vs-rest class counts use different kernels but the same likelihood rule. The training loss is the negative log probability assigned to the observed aggregate,

$$\mathcal{L}(\theta) = -\frac{1}{B} \sum_{b=1}^B \log (d_{b1} * \dots * d_{bn_b}) (y_b). \quad (12)$$

Gradients are backpropagated through FS-Conv into the instance network. Instance labels are never used for training in the aggregate-supervision experiments; they are used only for evaluation of hidden instance recovery. The expected-value baseline regresses only $\sum_i \mathbb{E}[z_i]$ to the observed aggregate, so it discards all information beyond the first moment.

Appendix A gives implementation details, including the classwise composite score used for one-vs-rest counts.

4.3 Special cases

Table 1 summarizes the task-specific kernels; FS-Conv itself is unchanged.

Binary counts. A target concept gives hidden Bernoulli indicators z_i , and the observed label is $Y = \sum_i z_i$. The kernel $[1 - p_i, p_i]$ recovers the standard Bernoulli count likelihood, so this case is included as a connection to prior count-loss work.

Signed counts. With observed signs $s_i \in \{-1, +1\}$, target instances contribute s_i and non-target instances contribute 0, giving $Y_{\pm} = \sum_i s_i z_i$. The signed kernels in Table 1 model cancellation explicitly because many latent assignments can yield the same signed sum.

Integer-valued sums. For integer-valued labels $z_i \in \{0, \dots, K - 1\}$, the atomic kernel is the full categorical vector and the observed label is $Y_{\text{sum}} = \sum_i z_i$. The likelihood therefore compares the observed sum to the full convolved PMF rather than only to the expected sum $\sum_i \mathbb{E}[z_i]$.

Histograms. For multiclass count labels, we use a classwise composite likelihood over one-vs-rest Bernoulli counts: for each class c , FS-Conv scores the observed count $h_c = \sum_i z_{ic}$, and the loss averages over classes. This is scalable, but it is not a full joint multiclass histogram model; it is closely related to the count-likelihood component of LLP-PVC. The corresponding composite score is therefore not a normalized joint histogram probability.

186 4.4 Computational backends

187 All of the above objectives are one-dimensional convolutions over integer aggregate support. If $m^{(i)}$
 188 denotes the aggregate PMF after processing the first i atoms, then

$$m^{(i)}(s) = \sum_{r=a}^b m^{(i-1)}(s-r) d_i(r), \quad m^{(0)} = \delta_0. \quad (13)$$

189 This recurrence is the same probability law whether implemented as dynamic programming, shifted
 190 dense tensor accumulation, grouped convolution, or FFT polynomial multiplication. The choice
 191 of backend is therefore computational rather than statistical: the likelihood being optimized is un-
 192 changed, but the hardware profile differs. We use sequential convolution for moderate binary, signed,
 193 and integer-valued sum experiments; Shukla-style dynamic programming as a Bernoulli count base-
 194 line; grouped GPU convolution for signed Bernoulli groups; and a balanced FFT-tree backend for
 195 large classwise one-vs-rest count PMFs. Further implementation details and backend complexity
 196 are given in Appendix A.

197 5 Experiments

198 5.1 Experimental setup

199 We evaluate FS-Conv under five aggregate-supervision themes:

- 200 1. integer-valued sum labels;
- 201 2. signed count labels with cancellation;
- 202 3. Bernoulli count labels as a recovery of prior count loss;
- 203 4. classwise count labels connecting to LLP;
- 204 5. runtime of exact aggregate backends.

205 Unless otherwise stated, MNIST-family experiments use LeNet-style instance networks, bags are
 206 sampled on the fly, and tables report mean $\pm$ sample standard deviation over random seeds when
 207 multiple seeds are available. Hidden instance labels are never used for training; they are used only
 208 for evaluation. Implementation details are summarized in Appendix A. Each subsection changes the
 209 observed aggregate label while keeping the same finite-support convolutional likelihood layer.

210 **Metrics.** Across the experimental tables, aggregate MAE columns report the mean absolute dif-
 211 ference between the predicted aggregate and the observed aggregate on test bags. For each seed
 212 and configuration, all methods are evaluated on the same deterministic held-out bag sampler; train
 213 and test bags are drawn from disjoint underlying dataset splits. For expected-value baselines, the
 214 predicted aggregate is the scalar expectation trained by the baseline. For FS-Conv rows, the main
 215 aggregate prediction is the mode of the learned aggregate PMF; we also report expected-aggregate
 216 MAE for the main FS-Conv settings where this statistic is directly comparable to expected-value
 217 baselines. For classwise count objectives, count MAE is the mean absolute class-count error aver-
 218 aged over classes and test bags,

$$\frac{1}{|\mathcal{B}_{\text{test}}|} \sum_{B \in \mathcal{B}_{\text{test}}} \frac{1}{C} \sum_{c=1}^C |\hat{h}_{Bc} - h_{Bc}|. \quad (14)$$

219 Rows labeled MSE denote MSE training objectives; MAE and accuracy columns are evaluation
 220 metrics computed after training. Columns labeled instance accuracy use held-out instance labels
 221 only for evaluation, never for training. Instance accuracy is never used for model selection unless
 222 explicitly stated.

223 5.2 Integer-valued sum labels

224 Digit-sum supervision is an integer-valued aggregate label: the learner observes a single scalar while
 225 each hidden instance label has ten possible values. Here z_i is the original MNIST digit value in

Table 2: Integer-valued sum labels. FS-Conv uses the full convolved PMF over sums, whereas MSE regresses only the expected sum. MNIST FS-Conv rows report mean $\pm$ standard deviation over three seeds; SVHN rows use ResNet-18 backbones and are single-seed diagnostics.

Data	n /train bags	FS-Conv MAE	FS-Conv inst. acc.	MSE MAE	MSE inst. acc.
MNIST	10/1000	0.612 $\pm$.008	.982 $\pm$.000	2.535	.226
MNIST	10/5000	0.277 $\pm$.026	.992 $\pm$.001	1.023	.746
MNIST	50/1000	4.901 $\pm$.973	.693 $\pm$.228	6.274	.248
MNIST	50/5000	2.229 $\pm$.616	.887 $\pm$.102	2.594	.817
SVHN scratch	10/5000	5.812	.229	5.328	.249
SVHN pretr.	10/5000	1.324	.955	2.109	.826

Table 3: Signed aggregates with random known signs. MSE regresses the expected signed sum $\sum_i s_i p_i$, while FS-Conv uses the full finite-support signed aggregate distribution. MNIST FS-Conv rows report mean $\pm$ standard deviation over three seeds; FashionMNIST rows report seed 0–1 means; SVHN and CIFAR-10 rows are seed-0 pretrained ResNet-18 experiments. A dash indicates that the corresponding paired MSE baseline was not evaluated.

Data	n /train bags	MSE MAE	MSE inst. acc.	FS-Conv MAE	FS-Conv inst. acc.
MNIST	10/1000	0.269	.959	0.095 $\pm$.010	.990 $\pm$.001
MNIST	10/5000	0.047	.995	0.045 $\pm$.010	.996 $\pm$.001
MNIST	50/1000	0.398	.991	0.243 $\pm$.053	.994 $\pm$.001
MNIST	50/5000	0.203	.995	0.198 $\pm$.022	.996 $\pm$.000
FashionMNIST	$n = 10$	0.121	.987	0.113	.988
FashionMNIST	$n = 50$	0.413	.988	0.381	.990
SVHN pretr.	$n = 16$	0.745	.926	0.684	.934
SVHN pretr.	$n = 64$	1.639	.914	1.407	.943
CIFAR-10 pretr.	$n = 16$	–	–	0.343	.974
CIFAR-10 pretr.	$n = 64$	–	–	0.753	.980

226 $\{0, \dots, 9\}$ and the aggregate label is $Y_{\text{sum}} = \sum_i z_i$. Exact finite-support likelihood uses the full
227 induced distribution over possible sums, whereas an expected-value baseline regresses only $\sum_i \mathbb{E}[z_i]$
228 to the observed sum.

229 For $n = 10$ and 1000 training bags, the MSE baseline has much worse aggregate error and hidden
230 digit recovery than FS-Conv. With more training bags it improves, but FS-Conv remains better in
231 both aggregate MAE and hidden digit accuracy. For $n = 50$, the task is harder for both objectives be-
232 cause many digit assignments can give similar sums; FS-Conv still has lower aggregate error, while
233 the lower hidden accuracy shows that large scalar-sum bags are underdetermined. SVHN supports
234 the same conclusion beyond MNIST when the visual backbone is strong: pretraining makes natural
235 RGB house-number digit sums usable, and FS-Conv outperforms expected-sum regression with the
236 same pretrained ResNet-18 backbone. On SVHN with 5000 bags, the pretrained FS-Conv model
237 reaches 95.46% hidden digit accuracy and 65.00% exact sum accuracy, while the pretrained MSE
238 baseline reaches 82.58% hidden digit accuracy and 22.58% exact sum accuracy. For the MNIST FS-
239 Conv rows, expected-aggregate MAE is 0.704 $\pm$.029, 0.306 $\pm$.022, 4.864 $\pm$ 1.003, and 2.183 $\pm$.544
240 for the four rows in Table 2, respectively.

241 5.3 Signed aggregates with cancellation

242 Signed counts test an aggregate that is not reducible to ordinary class proportions. We again use the
243 digit-9 target indicator, but each instance carries a known random sign; target-digit images contribute
244 +1 or −1, while all other digits contribute 0. Even with random signs, the observed signed sum is
245 many-to-one: positive and negative contributions can cancel, so different latent assignments can
246 yield the same aggregate label. FS-Conv models this full signed aggregate distribution, whereas the
247 MSE baseline only matches the expected signed sum.

248 Both methods recover accurate target detectors with enough data, but the likelihood uses the full
249 signed aggregate distribution rather than only $\sum_i s_i p_i$ and gives lower signed-count error. Fashion-
250 MNIST shows the same pattern beyond MNIST: MSE is close in instance recovery, but FS-Conv
251 is consistently better on the signed aggregate. On SVHN with ImageNet-pretrained ResNet-18, the

Table 4: Bernoulli count labels. The Bernoulli count likelihood recovers accurate hidden target detectors on MNIST target-count bags.

Data	n /train bags	Method	Count MAE	Instance acc.
MNIST	10/1000	exact count	0.115	0.988
MNIST	10/5000	exact count	0.090	0.991
MNIST	50/1000	exact count	0.686	0.987
MNIST	50/5000	exact count	0.460	0.991

Table 5: Main classwise count-supervision results. Count MAE is mean absolute class-count error. FS-Conv denotes the same finite-support convolutional likelihood layer applied as a classwise composite likelihood over one-vs-rest counts, as described in Section 4. A dash indicates that the corresponding paired MSE baseline was not evaluated.

Data	n /train bags	Backbone	MSE MAE	MSE inst. acc.	FS-Conv MAE	FS-Conv inst. acc.
MNIST	50/5000	LeNet	1.684 $\pm$.000	.116 $\pm$.000	.080 $\pm$.003	.992 $\pm$.000
CIFAR-10	16/1000	pretr. R18	.702	.668	.260	.902
CIFAR-10	64/1000	pretr. R18	1.855	.272	.678	.914
CIFAR-10	64/250	pretr. R18	—	—	1.115	.803

advantage is modest for $n = 16$ but clearer for $n = 64$, where FS-Conv improves instance accuracy from 91.37% to 94.33% and signed MAE from 1.639 to 1.407. The CIFAR-10 pretrained rows evaluate whether the signed finite-support objective remains usable in a harder natural-image setting; they are not paired with MSE baselines. They reach instance AUCs of 98.23% for $n = 16$ and 99.19% for $n = 64$. This setting illustrates a distinction from nonnegative count supervision: cancellation changes the inverse problem even when the latent instance variable remains Bernoulli. For the MNIST FS-Conv rows, posterior-mean signed MAE is 0.109 $\pm$.011, 0.050 $\pm$.006, 0.273 $\pm$.054, and 0.217 $\pm$.014 for the four rows in Table 3, respectively.

5.4 Bernoulli count likelihood as a special case

Bernoulli count supervision validates the connection to prior count-loss work. For MNIST-Bags, we make MNIST binary by choosing digit 9 as the positive class and treating digits 0–8 as negative. The learner observes the number of target-digit images in the bag; hidden instance labels are used only for evaluation. This case verifies that FS-Conv recovers the established Bernoulli count probability while sharing the same implementation pattern as the new aggregate types. Standard attention MIL is not a like-for-like baseline here, because it is typically trained from a binary presence label $\mathbf{1}\{Y > 0\}$, whereas this setting observes the full count Y .

This experiment is included as a recovery check rather than a novelty claim. It confirms that the same likelihood layer used for signed counts and integer-valued sums also gives the standard Bernoulli count likelihood and accurate hidden target recovery.

5.5 Classwise count objectives and LLP connections

We include histogram supervision to position FS-Conv relative to recent learning-from-label-proportions objectives. For MNIST histograms, the hidden category is the original digit class and the observed label is the vector of digit counts in the bag. We evaluate a classwise composite likelihood: for each class, the model scores the observed one-vs-rest count with a Bernoulli count PMF. This corresponds to the count-likelihood component of LLP-PVC, rather than a full joint multiclass histogram model. The comparison here is intentionally limited: MSE matches expected class counts, while the classwise FS-Conv likelihood scores each observed count with the exact one-vs-rest count PMF.

On the MNIST large-bag setting, classwise FS-Conv improves over MSE proportion matching in count MAE and hidden digit accuracy. This supports the connection to class-count LLP objectives without making the histogram setting the main novelty claim. The CIFAR-10 rows are narrower: they show that classwise FS-Conv is effective with ImageNet-pretrained ResNet-18 features and outperforms a pretrained MSE histogram baseline at $n = 16$ and $n = 64$. However, the $n = 64$,

Table 6: Runtime for multiclass one-vs-rest count PMFs with batch size 256 and $C = 10$ classes. CPU DP is compared to GPU FFT-tree convolution for forward aggregate PMF computation only.

Bag size	CPU DP (s)	GPU FFT-tree (s)	Speedup	GPU mem.	Max error
64	8.246	0.00097	8.5×10^3	13.8 MB	3.4×10^{-7}
128	17.230	0.00097	1.8×10^4	28.6 MB	4.2×10^{-7}
256	36.785	0.00121	3.0×10^4	55.6 MB	5.1×10^{-7}
512	54.040	0.00134	4.0×10^4	113.3 MB	6.6×10^{-7}

train 1000 setting exposes four times as many image instances per epoch as $n = 16$, train 1000. The fixed-instance-budget setting, $n = 64$ with 250 train bags, reaches only about 80.3% final instance accuracy. Thus the CIFAR-10 results should be read as evidence that FS-Conv can use aggregate-supervised image volume effectively under a strong representation, not as evidence that larger bags are intrinsically superior.

5.6 Backend runtime

Our FFT-tree backend computes batched multiclass one-vs-rest count PMFs on GPU and substantially reduces wall-clock time relative to a CPU dynamic-programming implementation in this benchmark. This addresses a practical limitation of DP-based count losses: exact count-probability computation need not be tied to sequential CPU dynamic programming. The benchmark in Table 6 isolates aggregate PMF evaluation for batch size 256 and $C = 10$ classes, so each timing evaluates 2560 binary count PMFs. It measures backend speed rather than end-to-end training time or representation quality.

6 Discussion

The experiments support FS-Conv as a common likelihood layer for finite-support aggregate supervision. Its strongest gains appear when the aggregate label is poorly summarized by a mean or proportion, as in low-data digit sums and signed counts with cancellation. The Bernoulli and class-wise count experiments primarily establish continuity with prior count-likelihood work: they show that the same interface includes prior count likelihoods and one-vs-rest count pseudo-likelihoods, not that ordinary count loss or full joint histogram LLP is new.

The main limitations are ambiguity and representation dependence. Large scalar-sum bags can admit many latent assignments with the same aggregate value, and signed aggregates make this non-identifiability explicit through cancellation. CIFAR-scale results also depend on pretrained features and sampled image exposure; the fixed-instance-budget comparison shows that larger bags are not intrinsically better.

Future work will use this finite-support signed-count view in the mathematical setting that motivated part of the project. Appendix B sketches how signed aggregate labels arise from conjectural factorizations of (q, t) -Littlewood–Richardson coefficients. Developing that application requires problem-specific architectures and diagnostics beyond the scope of this paper, but it is a natural next test of whether exact aggregate likelihoods can recover latent combinatorial structure.

References

- [1] Stuart Andrews, Ioannis Tsochantaridis, and Thomas Hofmann. Support vector machines for multiple-instance learning. In *Advances in Neural Information Processing Systems*, 2002.
- [2] Lorne Applebaum, Travis Dick, Claudio Gentile, Haim Kaplan, and Tomer Koren. Optimal learning from label proportions with general loss functions, 2025.
- [3] Gerda Bortsova, Florian Dubost, Silas Orting, Ioannis Katramados, Laurens Hogeweg, Laura Thomsen, Mathilde Wille, and Marleen de Bruijne. Deep learning from label proportions for emphysema quantification. *Medical Image Computing and Computer Assisted Intervention*, pages 768–776, 2018.

- [4] Robert Busa-Fekete, Heejin Choi, Travis Dick, Claudio Gentile, and Andres Munoz Medina. Easy learning from label proportions. In *Advances in Neural Information Processing Systems*, pages 14957–14968, 2023.
- [5] Robert Busa-Fekete, Travis Dick, Claudio Gentile, Haim Kaplan, Tomer Koren, and Uri Stemmer. Nearly optimal sample complexity for learning with label proportions. In *International Conference on Machine Learning*, 2025.
- [6] Marc-Andre Carbonneau, Veronika Cheplygina, Eric Granger, and Ghyslain Gagnon. Multiple instance learning: A survey of problem characteristics and applications. *Pattern Recognition*, 77:329–353, 2018.
- [7] James W. Cooley and John W. Tukey. An algorithm for the machine calculation of complex fourier series. *Mathematics of Computation*, 19(90):297–301, 1965.
- [8] Thomas G. Dietterich, Richard H. Lathrop, and Tomas Lozano-Perez. Solving the multiple instance problem with axis-parallel rectangles. *Artificial Intelligence*, 89(1–2):31–71, 1997.
- [9] Gabriel Dulac-Arnold, Neil Zeghidour, Marco Cuturi, Lucas Beyer, and Jean-Philippe Vert. Deep multi-class learning from label proportions, 2019.
- [10] Yen-Chang Hsu, Zhaoyang Lv, Joel Schlosser, Phillip Odom, and Zsolt Kira. Multi-class classification without multi-class labels. In *International Conference on Learning Representations*, 2019.
- [11] Maximilian Ilse, Jakub M. Tomczak, and Max Welling. Attention-based deep multiple instance learning. In *International Conference on Machine Learning*, pages 2127–2136, 2018.
- [12] Allen Knutson and Terence Tao. The honeycomb model of $gl_n(\mathbb{C})$ tensor products I: Proof of the saturation conjecture. *Journal of the American Mathematical Society*, 12(4):1055–1090, 1999.
- [13] D. E. Littlewood and A. R. Richardson. Group characters and algebra. *Philosophical Transactions of the Royal Society of London, Series A*, 233:99–141, 1934.
- [14] Jiabin Liu, Bo Wang, Zhiquan Qi, Yingjie Tian, and Yong Shi. Learning from label proportions with generative adversarial networks. In *Advances in Neural Information Processing Systems*, 2019.
- [15] Jiabin Liu, Bo Wang, Xin Shen, Zhiquan Qi, and Yingjie Tian. Two-stage training for learning from label proportions. In *International Joint Conference on Artificial Intelligence*, pages 2737–2743, 2021.
- [16] Tianhao Ma, Han Chen, Juncheng Hu, Yungang Zhu, and Ximing Li. Forming auxiliary high-confident instance-level loss to promote learning from label proportions. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 20592–20601, 2025.
- [17] Tianhao Ma, Wei Wang, Ximing Li, Gang Niu, and Masashi Sugiyama. Learning from label proportions via proportional value classification. In *International Conference on Learning Representations*, 2026.
- [18] I. G. Macdonald. *Symmetric Functions and Hall Polynomials*. The Clarendon Press, Oxford University Press, New York, second edition, 1995.
- [19] Oded Maron and Tomas Lozano-Perez. A framework for multiple-instance learning. In *Advances in Neural Information Processing Systems*, 1998.
- [20] Novi Quadrianto, Alex J. Smola, Tiberio S. Caetano, and Quoc V. Le. Estimating labels from label proportions. *Journal of Machine Learning Research*, 10:2349–2374, 2009.
- [21] Julien Schroeter, Kirill Sidorov, and David Marshall. Weakly-supervised temporal localization via occurrence count learning. In *International Conference on Machine Learning*, pages 5649–5659, 2019.

- [22] Clayton Scott and Jianxin Zhang. Learning from label proportions: A mutual contamination framework. In *Advances in Neural Information Processing Systems*, volume 33, pages 22256–22267, 2020.
- [23] Vinay Shukla, Zhe Zeng, Kareem Ahmed, and Guy Van den Broeck. A unified approach to count-based weakly supervised learning. In *Advances in Neural Information Processing Systems*, volume 36, pages 38709–38722, 2023.
- [24] Richard P. Stanley. Some combinatorial properties of Jack symmetric functions. *Advances in Mathematics*, 77:76–115, 1989.
- [25] Tao Sun, Dan Sheldon, and Brendan O’Connor. A probabilistic approach for learning with label proportions applied to the us presidential election. In *IEEE International Conference on Data Mining*, pages 445–454, 2017.
- [26] Kuen-Han Tsai and Hsuan-Tien Lin. Learning from label proportions with consistency regularization. In *Asian Conference on Machine Learning*, pages 513–528, 2020.
- [27] Ravi Vakil. A geometric Littlewood–Richardson rule. *Annals of Mathematics*, 164(2):371–422, 2006.
- [28] Xinggang Wang, Yongluan Yan, Peng Tang, Xiang Bai, and Wenyu Liu. Revisiting multiple instance neural networks. *Pattern Recognition*, 74:15–24, 2018.
- [29] Zixi Wei, Lei Feng, Bo Han, Tongliang Liu, Gang Niu, Xiaofeng Zhu, and Heng Tao Shen. A universal unbiased method for classification from aggregate observations. In *International Conference on Machine Learning*, pages 36804–36820, 2023.
- [30] Haoran Yang, Wanqing Zhang, and Wai Lam. A two-stage training framework with feature-label matching mechanism for learning from label proportions. In *Asian Conference on Machine Learning*, pages 1461–1476, 2021.
- [31] Felix X. Yu, Krzysztof Choromanski, Sanjiv Kumar, Tony Jebara, and Shih-Fu Chang. On learning from label proportions, 2014.
- [32] Jianxin Zhang, Yutong Wang, and Clayton Scott. Learning from label proportions by learning with label noise. In *Advances in Neural Information Processing Systems*, pages 26933–26942, 2022.

A Implementation details

All aggregate-supervision experiments sample bags on the fly rather than pregenerating fixed bag files. Within each seed and configuration, paired methods use the same deterministic held-out bag sampler so aggregate MAE and hidden-instance metrics are computed on the same test bags. Instance labels are used only for evaluation metrics such as hidden-instance AUC or hidden digit accuracy. MNIST-family experiments use LeNet-style instance classifiers, Adam with learning rate 5×10^{-4} , weight decay 10^{-4} , and 50 epochs. The main MNIST-family configurations use seeds 0, 1, 2, test on 1000 sampled bags, and evaluate mean bag sizes 10 and 50 with standard deviations 2 and 10.

Backend implementation. Sequential finite-support convolution accumulates shifted copies of the current aggregate PMF and is used for the moderate-size binary, signed, and integer-valued sum experiments. Dynamic programming is used as a Shukla-style Bernoulli count baseline and computes the same Bernoulli count coefficients as FS-Conv. For grouped signed binary aggregates, each active group starts as a point mass at zero on support $[-K_{\max}, K_{\max}]$. For one signed instance, the grouped GPU backend applies a length-three channelwise kernel and padding one. Because PyTorch’s conv1d operator is cross-correlation rather than mathematical convolution, the implementation stores the kernel in the order

$$[p_i \mathbf{1}\{s_i = +1\}, 1 - p_i, p_i \mathbf{1}\{s_i = -1\}], \quad (15)$$

Algorithm 1 Convolutional aggregate prediction for one minibatch

Require: Bags $\{(x_{b1:n_b}, a_{b1:n_b}, y_b)\}_{b=1}^B$, backbone f_θ , contribution map Φ , backend Conv**Ensure:** Loss $\mathcal{L}$ and aggregate-label scores $\{s_b\}_{b=1}^B$

```
1: for  $b = 1, \dots, B$  do
2:    $H_b \leftarrow f_\theta(x_{b1:n_b})$ 
3:   for  $i = 1, \dots, n_b$  do
4:      $d_{bi} \leftarrow \Phi(H_{bi}, a_{bi})$ 
5:   end for
6:   if classwise one-vs-rest count objective then
7:     for  $c = 1, \dots, C$  do
8:        $m_{bc} \leftarrow \text{Conv}(\{[1 - p_{bic}, p_{bic}]\}_{i=1}^{n_b})$ 
9:        $p_{bc} \leftarrow m_{bc}(h_{bc})$ 
10:    end for
11:     $s_b \leftarrow \prod_{c=1}^C p_{bc}^{1/C}$  ▷ composite score
12:     $\ell_b \leftarrow -C^{-1} \sum_{c=1}^C \log p_{bc}$ 
13:  else
14:     $m_b \leftarrow \text{Conv}(\{d_{bi}\}_{i=1}^{n_b})$ 
15:     $s_b \leftarrow m_b(y_b); \ell_b \leftarrow -\log s_b$ 
16:  end if
17: end for
18: return  $\mathcal{L} = B^{-1} \sum_{b=1}^B \ell_b, \{s_b\}_{b=1}^B$ 
```

Table 7: FS-Conv backend implementations. n is bag size, S is atomic support size, $T = n(S-1)+1$ is aggregate PMF width, and C is number of one-vs-rest classes.

Backend	Setting	Work per bag	Use
DP	Bernoulli count	$O(nT)$	Shukla-style count baseline
Sequential convolution	finite support	$O(nST)$	main moderate-bag experiments
Grouped convolution	signed Bernoulli	$O(nT)$, batched groups	signed GPU implementation
FFT-tree	finite support / one-vs-rest	$O(CT \log T \log n)$	runtime, classwise counts

415 so the left kernel entry shifts probability mass toward larger aggregate values and the right entry
416 shifts mass toward smaller values. This is only an implementation orientation; in mathematical
417 support order $[-1, 0, +1]$ the signed kernels are the atoms described in Section 4.

418 For digit-sum, the atom width is the number of finite digit values, for example ten for MNIST
419 digits. For class-count experiments, we avoid a full C -dimensional joint histogram over classes
420 and instead compute C one-vs-rest one-dimensional count likelihoods in parallel. This keeps the
421 aggregate support one-dimensional for each class and makes large bags practical on GPU. The
422 FFT-tree backend is useful for this setting because the same pairwise polynomial products are
423 batched over classes and bags. Padding or masked positions are represented by deterministic
424 zero-contribution atoms, such as $[1, 0]$ for Bernoulli counts, $[0, 1, 0]$ for signed support $[-1, 0, +1]$,
425 and $[1, 0, \dots, 0]$ for integer-valued sums. The main multiclass one-vs-rest runtime benchmark
426 is produced by `scripts/bench_multiclass_ovr_fft.py`; the reported CUDA run is stored
427 in `results/bench_multiclass_ovr_fft_cuda_20260505/`. This benchmark compares CPU
428 Bernoulli count DP with batched GPU FFT-tree convolution for forward aggregate PMF computa-
429 tion only, not full training.

430 These forward+backward measurements come from `scripts/bench_exactness_runtime.py`.
431 They show that GPU DP and convolution are close for simple binary counts, while the largest run-
432 time separation appears when many one-vs-rest PMFs are batched and evaluated by the FFT-tree
433 backend.

434 **Class-count and CIFAR settings.** Class-count experiments treat each class count as a one-vs-rest
435 Bernoulli count likelihood and optimize the resulting classwise composite likelihood. The MNIST
436 class-count experiments use batch size 192 for large-bag configurations. CIFAR-10 experiments use

Table 8: Representative forward+backward GPU runtime for exact aggregate likelihood backends.

Task	Bag size	Time (s)
Binary DP	32	0.0132
Binary convolution	32	0.0108
Binary DP	256	0.0963
Binary convolution	256	0.0749
Ordinal-10 sum convolution	32	0.0452
Ordinal-10 sum convolution	256	0.3449
Signed grouped convolution	32	0.00134
Signed grouped convolution	256	0.00416

ResNet-18 backbones. The pretrained one-vs-rest count settings use ImageNet-pretrained ResNet-18 with random crop and horizontal flip augmentation, SGD with momentum 0.9, weight decay 10^{-4} , learning rate 5×10^{-4} , and a cosine learning-rate schedule for 500 epochs. The pretrained CIFAR-10 analysis is limited to the evaluated $n = 16$ and $n = 64$ one-vs-rest count settings.

Compute resources. Reported GPU experiments were run on a single NVIDIA RTX 4090 GPU with 24 GB memory. The CPU dynamic-programming benchmark was run on the paired host CPU for that machine; the exact CPU model was not recorded in the experiment artifact. MNIST-family LeNet runs take approximately 0.4–1.5 GPU-hours per configuration, with peak memory ranging from about 0.4 GB for small-bag settings to 6.5 GB for the largest $n = 50$ settings. CIFAR-10 pretrained ResNet-18 runs take approximately 8–12 GPU-hours per configuration under the 500-epoch cosine schedule. The reported experiments and backend benchmarks use approximately 150 total GPU-hours; smoke tests and preliminary runs were used for development but are excluded from the reported tables.

B Mathematical origin: signed counting in (q, t) -Littlewood–Richardson coefficients

The signed-count instance of our framework was originally motivated by a problem in algebraic combinatorics: a conjectural factorization of the (q, t) -Littlewood–Richardson coefficients arising in the theory of Macdonald polynomials [18]. This object is central across several areas of mathematics—symmetric function theory, representation theory, Schubert calculus on Grassmannians, and partition combinatorics—and it also surfaces in mathematical physics through connections to the Calogero–Sutherland model and related integrable systems [13, 12, 27]. We sketch how this problem reduces to signed counting in our framework; complete mathematical exposition and empirical study are left for future work.

Setup. A partition $\lambda = (\lambda_1, \dots, \lambda_k)$ is identified with its Young diagram, a finite collection of unit cells. For a cell u in λ , the *arm length* $a_\lambda(u)$ is the number of cells strictly to the right of u in the same row, and the *leg length* $\ell_\lambda(u)$ is the number of cells strictly below u in the same column. Figure 2 illustrates these quantities on the partition $(4, 3, 1)$. The atomic building block of the theory is the rational function

$$b(a, \ell) := \frac{1 - q^a t^{\ell+1}}{1 - q^{a+1} t^\ell}, \quad b_\lambda(u; q, t) := b(a_\lambda(u), \ell_\lambda(u)). \quad (16)$$

The conjectural factorization. The Littlewood–Richardson coefficient $c_{\lambda, \mu}^\nu \in \mathbb{Z}_{\geq 0}$ [13] admits a (q, t) -refinement $f_{\lambda, \mu}^\nu(q, t)$, a rational function in q, t with $f_{\lambda, \mu}^\nu(0, 0) = c_{\lambda, \mu}^\nu$. A long-standing conjecture of Stanley [24] predicts that whenever $c_{\lambda, \mu}^\nu = 1$, there exist subsets $X_{\lambda, \mu}^\nu \subseteq \lambda$, $Y_{\lambda, \mu}^\nu \subseteq \mu$, and $Z_{\lambda, \mu}^\nu \subseteq \nu$ such that

$$f_{\lambda, \mu}^\nu(q, t) = \frac{\prod_{u \in Z_{\lambda, \mu}^\nu} b_\nu(u; q, t)}{\prod_{u \in X_{\lambda, \mu}^\nu} b_\lambda(u; q, t) \cdot \prod_{u \in Y_{\lambda, \mu}^\nu} b_\mu(u; q, t)}. \quad (17)$$

The conjecture remains open in general; the case where μ is a single-row partition is known explicitly as the *Pieri rule* [18, p. 340].

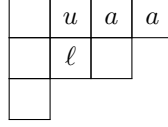


Figure 2: Young diagram of the partition $(4, 3, 1)$. For the cell $u = (1, 2)$, the arm length $a_\lambda(u) = 2$ counts the cells marked a to the right of u in the same row, and the leg length $\ell_\lambda(u) = 1$ counts the cell marked ℓ below u in the same column.

Reduction to signed finite-support aggregate likelihoods. Two observations connect Eq. (17) to signed aggregate counting. First, the atomic factor $b_\lambda(u; q, t)$ depends on the cell u only through its arm-leg pair $(a_\lambda(u), \ell_\lambda(u))$, so cells from different partitions or different positions but sharing the same value (a, ℓ) all contribute the same factor $b(a, \ell)$ to $f_{\lambda, \mu}^\nu(q, t)$, and may therefore be grouped together. Second, the conjectural subsets enter Eq. (17) with opposite multiplicative roles: cells in $Z_{\lambda, \mu}^\nu \subseteq \nu$ appear in the numerator, while cells in $X_{\lambda, \mu}^\nu \subseteq \lambda$ and $Y_{\lambda, \mu}^\nu \subseteq \mu$ appear in the denominator. Letting $+1$ (resp. -1) signify a numerator (resp. denominator) contribution and grouping cells by arm-leg type, the multiplicities collapse into a single signed integer exponent per type, yielding

$$f_{\lambda, \mu}^\nu(q, t) = \prod_{(a, \ell) \in \mathbb{Z}_{\geq 0}^2} b(a, \ell)^{\beta_{\lambda, \mu, \nu, a, \ell}}, \quad \beta_{\lambda, \mu, \nu, a, \ell} = \sum_{u \in \mathcal{B}_{\lambda, \mu, \nu, a, \ell}} s_u z_u. \quad (18)$$

Here $\mathcal{B}_{\lambda, \mu, \nu, a, \ell} = \{u \in \lambda \sqcup \mu \sqcup \nu : (a_u, \ell_u) = (a, \ell)\}$ is the arm-leg bag, the sign is $s_u = +1$ for $u \in \nu$ and $s_u = -1$ for $u \in \lambda \sqcup \mu$, and $z_u \in \{0, 1\}$ indicates whether the cell u belongs to the conjectural selected subset $X_{\lambda, \mu}^\nu \cup Y_{\lambda, \mu}^\nu \cup Z_{\lambda, \mu}^\nu$. The exponents $\beta_{\lambda, \mu, \nu, a, \ell}$ are read off from the rational expression of $f_{\lambda, \mu}^\nu(q, t)$, while the underlying selections $\{z_u\}$ are unobserved and need not be unique: distinct latent selections can produce the same vector of observed exponents, exactly by the cancellation phenomenon that distinguishes signed counting from plain Bernoulli counting.

Identifying instances with cells, bags with arm-leg equivalence classes, the aggregate label with the observed exponent $\beta_{\lambda, \mu, \nu, a, \ell}$, and the latent selection with membership in the conjectural subset, Eq. (18) is exactly the signed finite-support aggregate-likelihood setting of Section 3. Each triple (λ, μ, ν) produces a family of such bags indexed by (a, ℓ) , each with a signed integer label, and the convolutional bag-prediction layer of the main paper applies without modification.

Empirical note. This signed-count view suggests a future experiment: learning latent cell selections from aggregate exponents under the conjectural factorization. The detailed mathematical formulation, problem-specific architectural choices, and combinatorial diagnostics are outside the scope of the present paper and are left to a separate report.

NeurIPS Paper Checklist

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: [Yes]

Justification: The claims in the abstract and Section 1 are supported by Sections 3, 4, and 5, and Section 6 states scope limitations to avoid overclaiming.

Guidelines:

- The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
- The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
- The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
- It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [Yes]

Justification: Section 6 discusses limitations including aggregate ambiguity, signed-aggregate cancellation, representation dependence, and the CIFAR-10 fixed-instance-budget caveat.

Guidelines:

- The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate “Limitations” section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren’t acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [Yes]

Justification: Definition 1 and Eq. (6) state the conditional-independence assumptions explicitly; the likelihood follows from the standard convolution rule for sums of independent integer-valued random variables.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: Section 5 and Appendix A specify the datasets, backbones, optimizer, bag-size distribution, and seeds used for the main results.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general, releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
 - (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
 - (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
 - (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
 - (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [Yes]

Justification: Anonymized code with instructions to reproduce the main experimental results is provided as supplementary material.

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [Yes]

Justification: Section 5 and Appendix A list the data splits, optimizer, hyperparameters, batch size, epochs, and augmentation used in each setting.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [No]

Justification: Section 5 captions specify the seed count for each configuration, but most main tables report point estimates rather than full error bars or confidence intervals.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.

- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [Yes]

Justification: Appendix A reports the GPU type, memory, approximate per-configuration runtime ranges, and total GPU-hour estimate.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. Code of ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines>?

Answer: [Yes]

Justification: The work uses standard public image-classification benchmarks and involves no human subjects or sensitive data, conforming with the NeurIPS Code of Ethics.

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. Broader impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [N/A]

Justification: The work is foundational methodology for aggregate-supervised MIL and is not tied to a specific application or deployment with identifiable societal impact.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. Safeguards

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [N/A]

Justification: No new high-risk models or datasets are released; only standard public benchmarks and architectures are used.

Guidelines:

- The answer [N/A] means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [Yes]

Justification: The datasets and pretrained backbones used are publicly available standard research benchmarks cited in the references; their original creators are credited.

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.

- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, paperswithcode.com/datasets has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. New assets

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [Yes]

Justification: The supplementary code includes an anonymized README with setup instructions, basic test commands, and pointers to the provided scripts and configurations.

Guidelines:

- The answer [N/A] means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. Crowdsourcing and research with human subjects

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [N/A]

Justification: The paper does not involve crowdsourcing or research with human subjects.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. Institutional review board (IRB) approvals or equivalent for research with human subjects

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [N/A]

Justification: The paper does not involve research with human subjects.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.

- 808 • Depending on the country in which research is conducted, IRB approval (or equivalent)
809 may be required for any human subjects research. If you obtained IRB approval,
810 you should clearly state this in the paper.
- 811 • We recognize that the procedures for this may vary significantly between institutions
812 and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the
813 guidelines for their institution.
- 814 • For initial submissions, do not include any information that would break anonymity
815 (if applicable), such as the institution conducting the review.

816 16. Declaration of LLM usage

817 Question: Does the paper describe the usage of LLMs if it is an important, original, or
818 non-standard component of the core methods in this research? Note that if the LLM is used
819 only for writing, editing, or formatting purposes and does *not* impact the core methodology,
820 scientific rigor, or originality of the research, declaration is not required.

821 Answer: [N/A]

822 Justification: The core methodology does not involve LLMs.

823 Guidelines:

- 824 • The answer [N/A] means that the core method development in this research does not
825 involve LLMs as any important, original, or non-standard components.
- 826 • Please refer to our LLM policy in the NeurIPS handbook for what should or should
827 not be described.